



Universidade de Brasília

Departamento de Ciência da Computação

RISC-V Uniciclo

Unidade de Controle



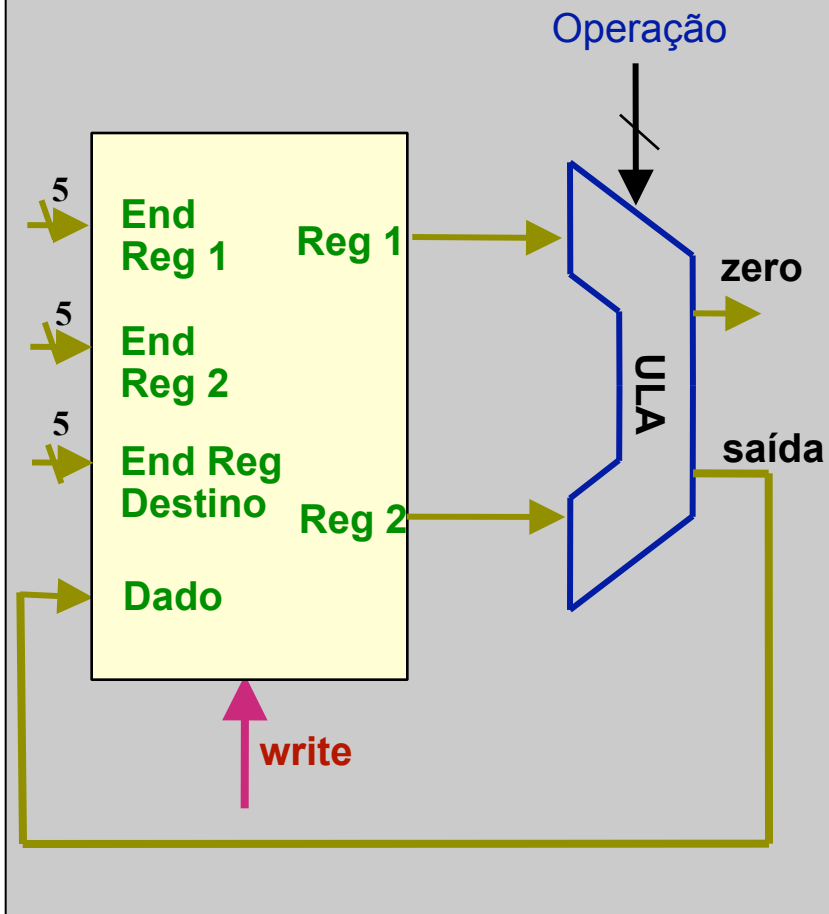
RISC-V Uniciclo

- Foram desenvolvidas, na verdade, três tipos de unidades operativas:
 - uma para instruções no formato R (*add*, *sub*, etc.)
 - uma para instruções de acesso a memória *lw* e *sw* - formato I
 - uma para *instruções condicionais (beq)* – formato I
- Na fase de projeto às vezes precisamos replicar recursos.
- Na via de dados mais simples deve-se propor executar as instruções em um único período do *clock*.
- Isto quer dizer que nenhum dos recursos pode ser usado mais de uma vez por instrução.

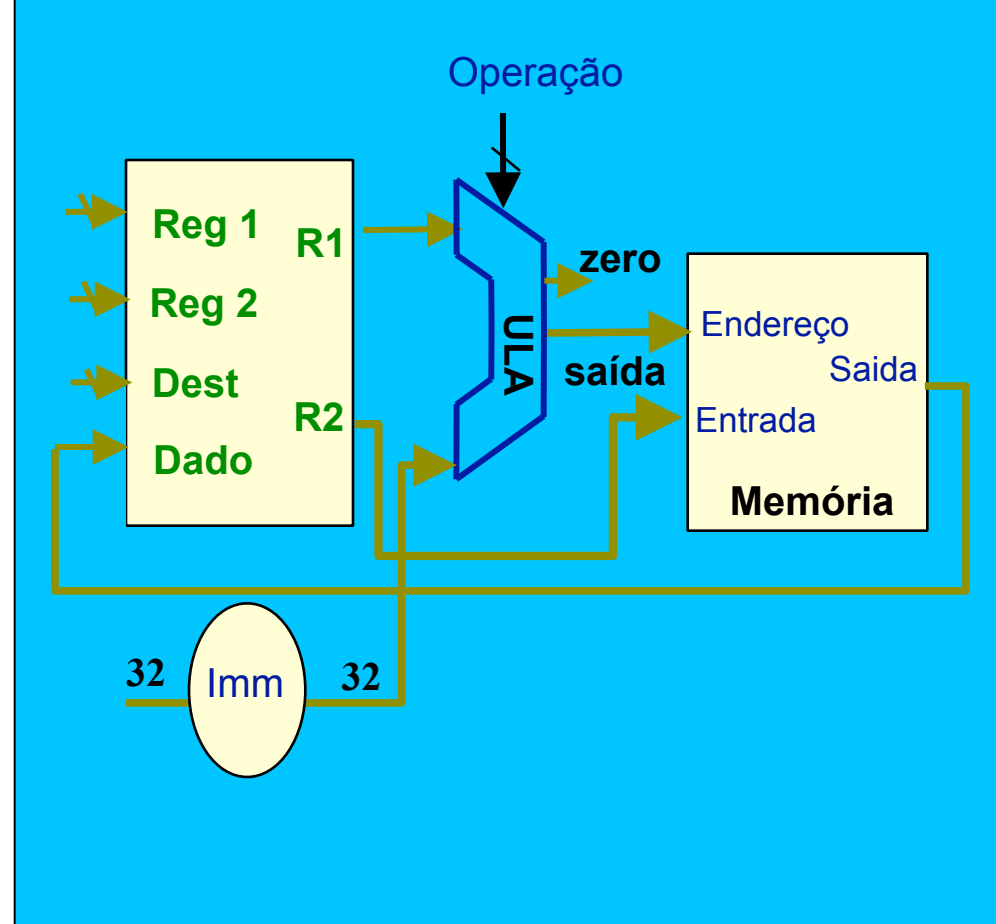


Instruções lóg/arit (tipo-R) e lw/sw

Lógico-aritméticas

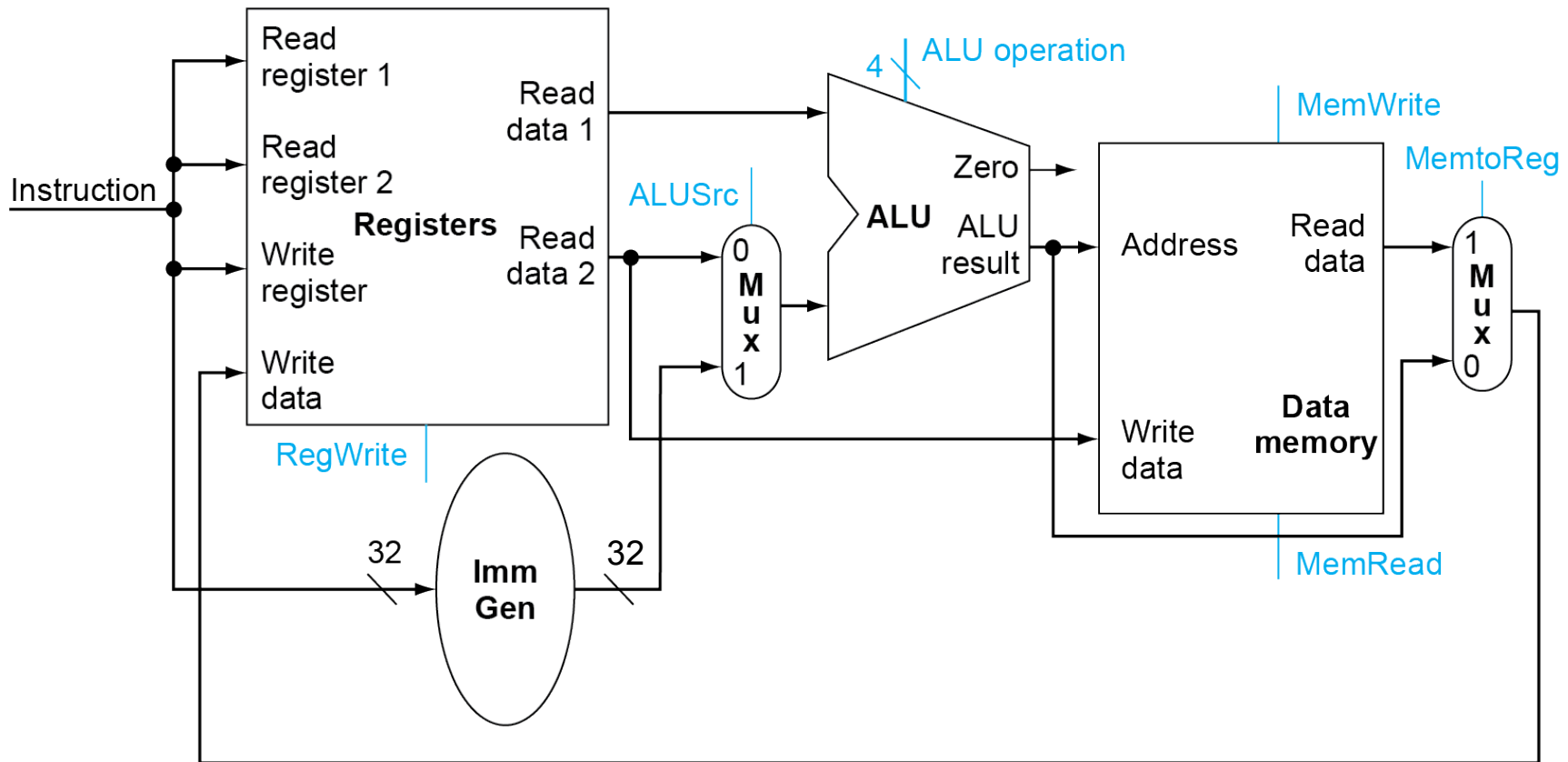


Leitura-escrita



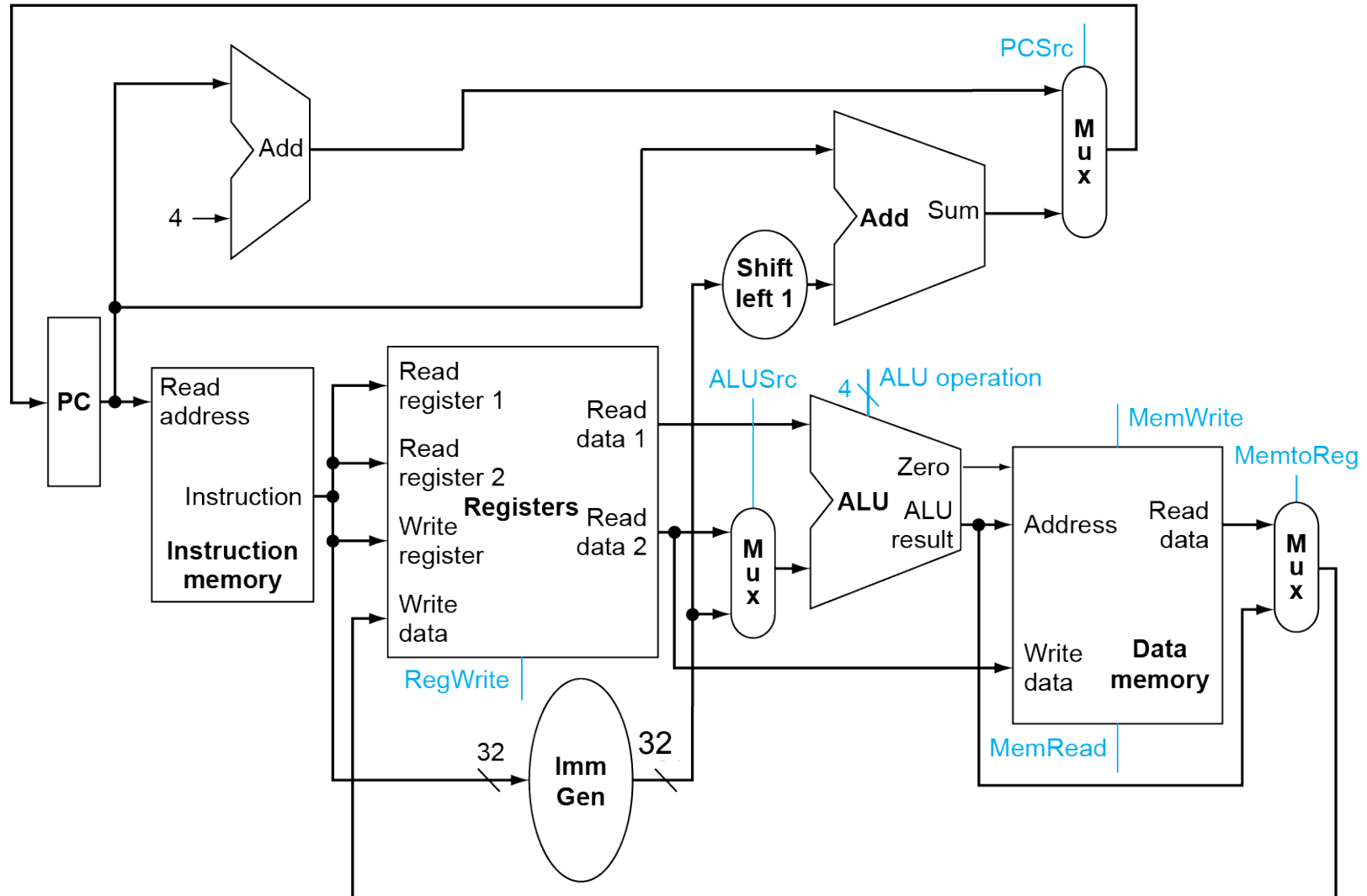


Log-Aritméticas / Load / Store





Unidade Operativa: Quase-Final





Controle do RISC-V

- Cada operação requer a utilização adequada dos recursos do RISC-V
- Ex: *add t0, s1, s2*
 - é necessário que os endereços dos operandos *s1* e *s2* sejam enviados ao banco de registradores
 - Da mesma maneira, o endereço do registrador destino (*t0*) deverá ser informado ao banco de registradores
 - Uma vez que o banco de registradores disponibilize os valores de *s1* e *s2*, estes deverão ser encaminhados à ULA
 - Posteriormente, o resultado deverá ser escrito no banco de registradores (registrador *t0*)



Controle da ULA

- As operações da ULA são controladas por um código de 4 bits:

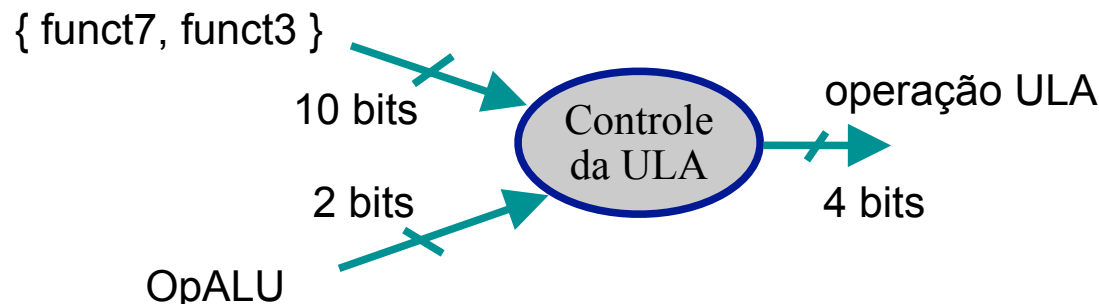
Linhas de Controle ULA	Função
0000	AND
0001	OR
0010	ADD
0110	SUB
0111	SLT

- As instruções **load** e **store** utilizam a operação de **soma** da ULA
- As instruções do **tipo R** utilizam uma das **5 operações**
- A instrução **beq** utiliza a **subtração** da ULA



Identificação da Operação

- Os Opcodes definem as operações:
lw(0x03), sw(0x23), beq(0x63), Tipo-R (0x33)
- No Tipo-R os campos funct7 e funct3, especificam o tipo de operação aritmética/lógica.
- Vamos definir 2 bits para identificar cada uma das categorias de instruções (opALU):
 - 00 acesso à memória (lw, sw) [**add**]
 - 01 desvio (beq, bne, blt, bge, bltu, bgeu)
 - 10 lógico-aritméticas ([**add**, **sub**, **and**, **or**, **slt**])





Circuito de controle da ULA

- 'x' indica irrelevâncias (*don't cares*)

OPcode	ALUOp	Operação	Funct7 Funct3	Operação da ULA	ALU Control
lw	00	Load	xxxxxxx xxx	add	0010
sw	00	Store	xxxxxxx xxx	add	0010
beq	01	Beq	xxxxxxx xxx	sub	0110
Tipo-R	10	Add	0000000 000	add	0010
Tipo-R	10	Sub	0100000 000	sub	0110
Tipo-R	10	And	0000000 111	and	0000
Tipo-R	10	Or	0000000 110	or	0001
Tipo-R	10	Slt	0000000 010	slt	0111



Tabela Verdade Controle ULA

- Uma tabela simplificada com irrelevâncias. Pode-se derivar um circuito lógico que gera as 4 saídas

ALUOp		Funct 7							Funct 3			ALU Control
ALUOp _{p1}	ALUOp _{p0}	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	Operação
0	0	x	x	x	x	x	x	x	x	x	x	0 0 1 0
0	1	x	x	x	x	x	x	x	x	x	x	0 1 1 0
1	0	0	0	0	0	0	0	0	0	0	0	0 0 1 0
1	0	0	1	0	0	0	0	0	0	0	0	0 1 1 0
1	0	0	0	0	0	0	0	0	1	1	1	0 0 0 0
1	0	0	0	0	0	0	0	0	1	1	0	0 0 0 1
1	0	0	0	0	0	0	0	0	0	1	0	0 1 1 1



```

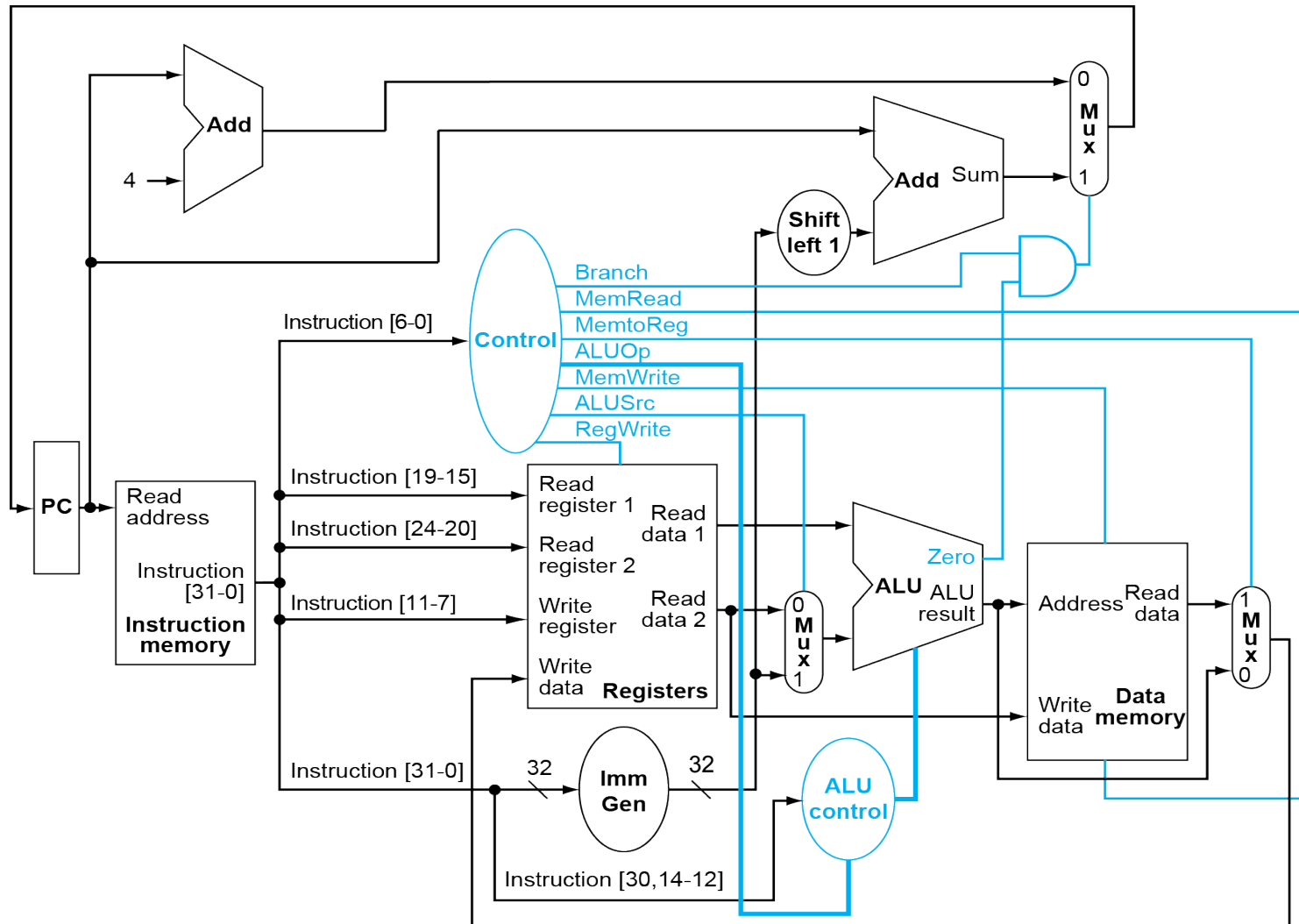
entity cntrULA is
port ( funct7   : in std_logic_vector(6 downto 0);
       funct3    : in std_logic_vector(2 downto 0);
       aluop     : in std_logic_vector(1 downto 0);
       aluctr    : out std_logic_vector(3 downto 0));
end entity;
architecture arch of cntrULA is
begin
process(funct7, funct3, aluop) begin
    case aluop is
        when "00" => aluctr <= "0010";  -- soma
        when "01" => aluctr <= "0110";  -- subtração
        when "10" => case funct3 is
            when "000" => if (funct7(5)='1')
                           then aluctr <= "0010";
                           else aluctr <= "0110";
                           end if;
            when "111" => aluctr <= "0000";
            when "110" => aluctr <= "0001";
            when "010" => aluctr <= "0111";

        end case;
    end process;
end architecture;

```



Controle Uniciclo





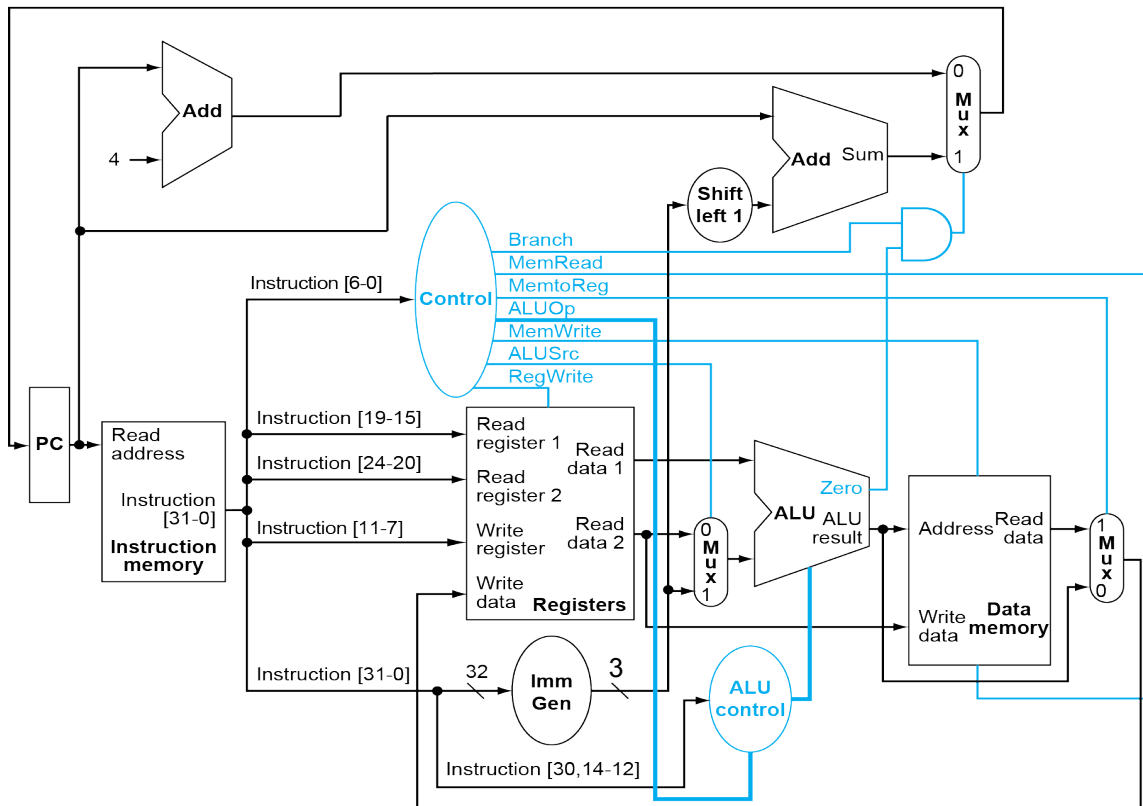
Entrada da Unidade de Controle

- as informações necessárias a execução de uma instrução são retiradas da própria instrução
 - O *opcode* (código de operação) sempre está nos bits $RI[6:0]=Op[6:0]$
 - Os 2 registradores a serem lidos (*rs1* e *rs2*) sempre estão nas posições $RI[19:15]$ e $RI[24:20]$ (tipo-R, beq e sw)
 - O registrador destino (*rd*) está sempre na posição $RI[11:7]$
 - O dado Imediato depende da instrução



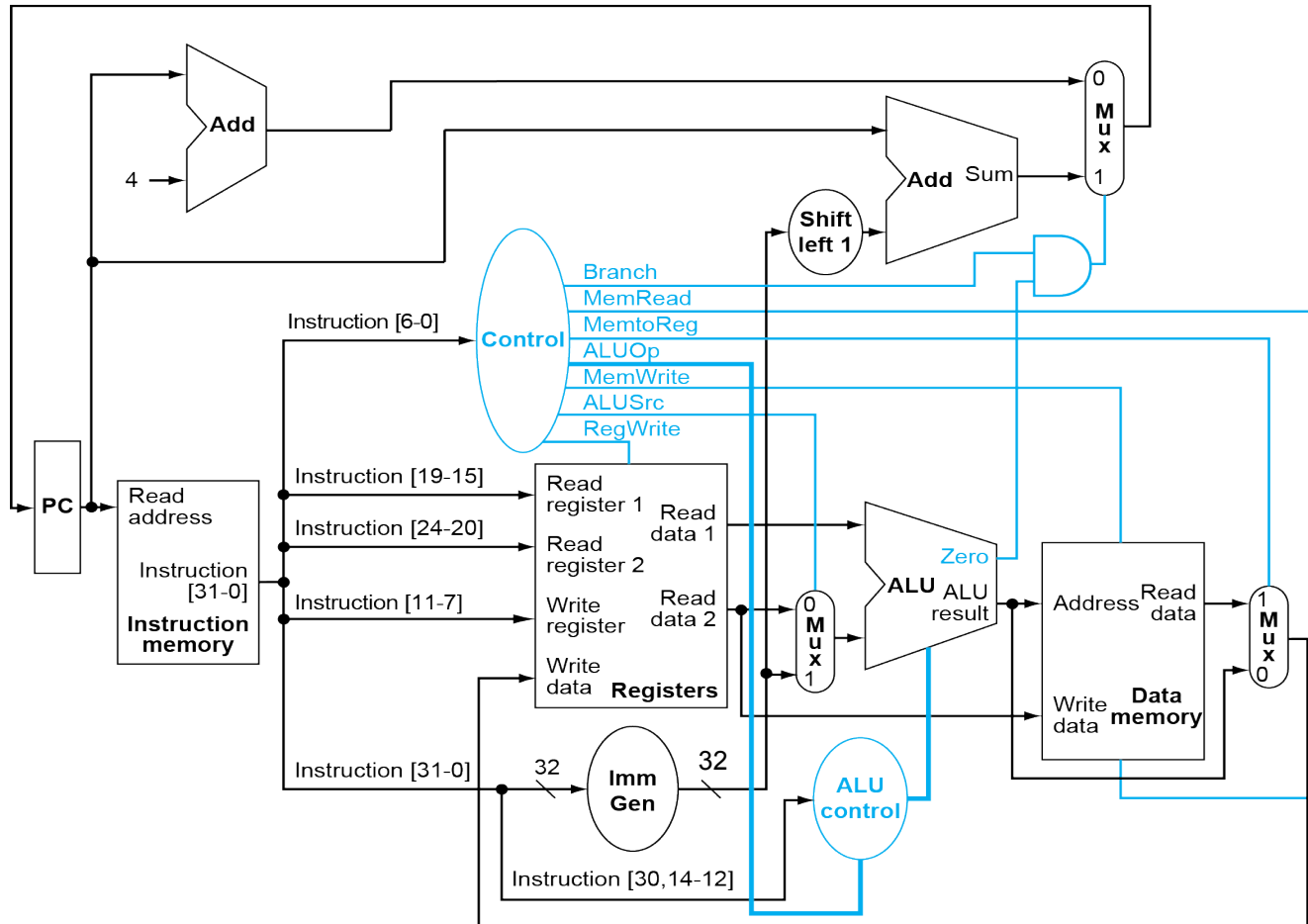
Sinais de Controle

- A unidade de controle de prover:
 - sinais para os multiplexadores
 - sinais de leitura e escrita para as memórias
 - seleção da operação da ULA
 - controle do novo endereço a ser carregado no PC, para instruções de salto





Sinais de Controle



Instrução	ALUSrc	Mem2Reg	RegWrite	MemRead	MemWrite	Branch	ALUOp
Tipo-R	0	0	1	0	0	0	10
lw	1	1	1	1	0	0	00
sw	1	X	0	0	1	0	00
beq	0	X	0	0	0	1	01



Tabela Verdade do Controle

Instrução	Opcode	ALUSrc	Mem2Reg	RegWrite	MemRead	MemWrite	Branch	ALUOp
Tipo-R	0110011	0	0	1	0	0	0	10
lw	0000011	1	1	1	1	0	0	00
sw	0100011	1	X	0	0	1	0	00
beq	1100011	0	X	0	0	0	1	01

■ Implementações

- lógica combinacional (síntese lógica)
- ROM
- PLAs (soma de produtos)
 - lógica AND decodifica a instrução
 - lógica OR identifica os acionamentos:
 - ex: $\text{RegWrite} = (\text{Tipo-R OR lw})$



Exercícios

- 1) Analisar a instrução: add t1,t2,t3
- 2) Analisar a instrução: lw t1,offset(t2)
- 3) Analisar a instrução: beq t1,t2,offset
- 4) Estender a organização do RISC-V para dar suporte a execução da instrução **JAL**, Jump And Link, desvio incondicional
O endereço de desvio é obtido por: $PC + \text{Imediato} \ll 1$



Problemas com RISC-V Uniciclo

- Período do relógio determinado pelo caminho mais longo
 - **instrução lw:** (imagina se tivesse operações em Float Point!)
 - leitura da instrução
 - leitura do registrador de base, extensão de sinal
 - cálculo do endereço
 - leitura do dado da memória
 - escrita em registrador
- TODAS as instruções levam o mesmo tempo para executar: CPI=1

E ainda!

- Viola o princípio de tornar o caso comum rápido!
- Unidades funcionais duplicadas



Análise de Desempenho Uniciclo

- Supondo os seguintes tempos de execução das unidades operativas:
 - Acesso a memória: 200ps
 - ULA e somadores: 100ps
 - Acesso ao banco de registradores (leitura ou escrita): 50ps
 - Multiplexadores, Unidade de Controle, acesso ao PC, Unidade de Extensão de Sinal e fios considerados sem atraso (!!!!)
- Qual das seguintes implementações seria mais rápida e quanto?
 - 1) Implementação onde toda instrução é feita em 1 ciclo de clock de duração fixa.
 - 2) Implementação onde toda instrução é feita em 1 ciclo de clock de duração somente o necessário (!!**exemplo ideal - didático**!!)

Para comparação consideremos um workload composto de:
25% loads, 10% stores, 45% ALU, 15% desvios condicionais, 5% jumps



■ Lembrando que:

Tempo Execução = Contagem de Instruções x CPI x Período de clock

Classe de instrução	Unidades funcionais usadas pela classe de instrução				
tipo R	Busca de instrução	Acesso a registrador	ALU	Acesso a registrador	
Load word	Busca de instrução	Acesso a registrador	ALU	Acesso à memória	Acesso a registrador
Store word	Busca de instrução	Acesso a registrador	ALU	Acesso à memória	
Branch	Busca de instrução	Acesso a registrador	ALU		
Jump	Busca de instrução				

Usando esses caminhos críticos, podemos calcular o tamanho exigido para cada classe de instrução:

Classe de instrução	Memória de instrução	Leitura de registrador	Operação ALU	Memória de dados	Escrita de registrador	Total
Tipo R	200	50	100	0	50	400ps
Load word	200	50	100	200	50	600ps
Store word	200	50	100	200		550ps
Branch	200	50	100	0		350ps
Jump	200					200ps

Tempo médio:

Uniciclo fixo: 600ps

Uniciclo variável: $600 \times 0.25 + 550 \times 0.1 + 400 \times 0.45 + 350 \times 0.15 + 200 \times 0.05 = 447.5\text{ps}$

Fator de desempenho: $n = 600 / 447.5 = 1.34$